



**Документация на курсов проект по
Програмни Среди**

Александър Евгениев Ганчев

ФКСТ, КСИ, група 39а

№ 121223237



Съдържание

1. Въведение.....	2
2. Спецификация на изискванията.....	3
3. Проектиране на системата.....	9
4. Проектиране на базата данни.....	16
5. Имплементация.....	21



1. Въведение

GitHub линк към проекта:

<https://github.com/AleksandarGanchevDev/NoteTaker>

Проектът представлява приложение за водене на бележки, реализирано чрез **.NET MAUI**. То е предназначено за създаване, редактиране и организиране на бележки в отделни папки, като предоставя възможност за работа с **Markdown** съдържание, визуализация на неговия форматиран текст, съхраняване на предишни версии и използване на **undo/redo**. Допълнително е реализирана и функционалност за **drag-and-drop** преместване на бележки между папки, което улеснява организирането на бележките.

Проектът използва архитектурата **MVVM (Model-View-ViewModel)**, който осигурява ясно разделение между потребителския интерфейс, бизнес логиката и моделите на данните. За съхранение на информацията е използвана **SQLite** база данни, което позволява приложението да работи самостоятелно, без необходимост от външен сървър или постоянна интернет връзка.

Основната цел на проекта е да се създаде функционално, добре структурирано и лесно за използване приложение, което да демонстрира практическо приложение на изучаваните технологии и принципи за разработка на софтуерни системи. В рамките на разработката са реализирани както основни функционалности, свързани с управление на бележки и папки, така и допълнителни възможности, които повишават качеството и завършеността на проекта.

В следващите раздели на документацията са разгледани изискванията към системата, използваните технологии, архитектурата на приложението, базата данни, реализираните функционалности, както и основните диаграми.



2. Спецификация на изискванията

В тази точка ще опиша основните изисквания към разработеното приложение. Те се разделят на **функционални**, свързани с конкретните възможности на приложението, и **нефункционални**, които определят изискванията към нейната работа, надеждност и удобство. В края на главата са представени основните **use cases**, които описват взаимодействието на потребителя със системата.

2.1. Функционални изисквания

Функционалните изисквания определят конкретните действия и операции, които системата трябва да поддържа.

- **Системата трябва да позволява създаване на нови папки**
Потребителят трябва да може да въвежда име на папка и да я добавя, като тя се записва в базата данни и става достъпна в списъка с папки.
- **Системата трябва да позволява изтриване на папки**
Потребителят трябва да може да изтрие избрана папка. При изтриване на папка, бележките в нея не трябва да се губят, а да се преместват в друга съществуваща папка. Системата не трябва да позволява изтриване на последната останала папка.
- **Системата трябва да позволява избор на активна папка**
Потребителят трябва да може да избира папка от списъка, след което да се зареждат бележките, принадлежащи към нея.
- **Системата трябва да позволява създаване на нови бележки**
Потребителят трябва да може да създава нова бележка в рамките на текущо избраната папка
- **Системата трябва да позволява редактиране на бележки**
Потребителят трябва да може да променя заглавието и съдържанието на избрана бележка.
- **Системата трябва да позволява записване на бележки в база данни**
Всички промени по бележките трябва да могат да бъдат записвани в SQLite база данни, така че информацията да се съхранява между отделните стартирания на приложението



- **Системата трябва да поддържа Markdown редактор**
Потребителят трябва да може да въвежда съдържание в Markdown формат, включително заглавия, списъци, удебелен текст и други стандартни елементи
- **Системата трябва да визуализира Markdown текста**
Въведеното Markdown съдържание трябва да бъде преобразувано и визуализирано в отделен панел като форматиран преглед
- **Системата трябва да съхранява ревизии на бележките**
При записване на промяна в съдържанието на съществуваща бележка, предишната версия трябва да бъде съхранена като ревизия в базата данни
- **Системата трябва да позволява преглед на ревизии на избраната бележка**
Потребителят трябва да може да вижда списък с минали версии на текущо избраната бележка
- **Системата трябва да позволява възстановяване на ревизия**
Потребителят трябва да може да избере стара версия и да я зареди обратно в редактора, след което да я запише като текуща версия на бележката
- **Системата трябва да поддържа undo/redo механизъм**
По време на редакция потребителят трябва да може да отменя последната направена промяна и да я възстановява отново
- **Системата трябва да поддържа drag-and-drop преместване на бележки между папки**
Потребителят трябва да може да прехвърля бележка от една папка в друга чрез влачене и пускане върху желаната папка
- **Системата трябва да позволява изтриване на бележки**
Потребителят трябва да може да изтрие избрана бележка
- **Системата трябва да визуализира статусна информация**
Приложението трябва да предоставя кратки съобщения към потребителя за успешно записване, изтриване, преместване или възникнали ограничения

2.2. Нефункционални изисквания

Нефункционалните изисквания определят качествата, които системата трябва да притежава, за да бъде надеждна и удобна.



- **Удобство при работа**
Потребителският интерфейс трябва да бъде ясен, подреден и интуитивен, така че основните действия да могат да се извършват лесно и бързо.
- **Локално съхранение на данните**
Приложението трябва да работи без необходимост от интернет връзка, като всички данни се съхраняват локално в SQLite база данни.
- **Надеждност на съхранението**
След записване на бележка, папка или ревизия данните трябва да бъдат достъпни и след повторно стартиране на приложението.
- **Добра производителност**
Приложението трябва да реагира бързо при зареждане на папки, бележки и ревизии, както и при запис и изтриване.
- **Поддържаемост на кода**
Системата трябва да бъде изградена с ясна структура и разделение на отговорностите, така че кодът да бъде лесен за разширяване и поддръжка.
- **Спазване на архитектурен шаблон**
Реализацията на приложението трябва да следва шаблона MVVM, като се поддържа ясно разделение между View, ViewModel, Model и достъпа до данни.

2.3. Случаи на употреба (Use Cases)

При use case-ове, актьорът представлява външен участник, който взаимодейства със системата и инициира определени действия в нея. Случаите на употреба (use cases) описват основните функционалности на системата от гледна точка на този участник, тоест какви операции може да извършва той при работа с приложението.

2.3.1. Основен актьор

- **Потребител** - лице, което използва приложението за създаване, организиране и управление на бележки.

2.3.2. Основни случаи на употреба

- **Създаване на папка**
Потребителят въвежда име на нова папка и избира команда за



добавяне. Системата валидира входните данни, записва папката в базата и обновява списъка с папки.

- **Изтриване на папка**

Потребителят избира папка и стартира операция по изтриване.

Системата изисква потвърждение, премества бележките от папката в друга съществуваща папка и след това изтрива папката.

- **Избор на папка**

Потребителят избира папка от списъка. Системата зарежда всички бележки, които принадлежат към нея.

- **Създаване на бележка**

Потребителят избира команда за създаване на нова бележка.

Системата подготвя празен редактор и при запис създава нов запис в базата данни.

- **Редактиране и записване на бележка**

Потребителят въвежда или променя заглавието и съдържанието на бележка. При запис системата съхранява промените в базата данни.

- **Преглед на Markdown preview**

Потребителят редактира съдържанието на бележката в Markdown формат. Системата визуализира форматиран preview в отделен панел.

- **Създаване на ревизия**

При записване на променена бележка системата автоматично създава ревизия със старото съдържание.

- **Преглед и възстановяване на ревизия**

Потребителят избира ревизия от списъка. Системата зарежда съдържанието ѝ в редактора, откъдето то може да бъде записано отново като актуална версия.

- **Undo/Redo по време на редакция**

Потребителят извършва редакции в съдържанието и използва бутоните Undo и Redo за връщане към предишно състояние или повторно прилагане на промяната.

- **Преместване на бележка чрез drag-and-drop**

Потребителят влачи бележка от списъка с бележки и я пуска върху друга папка. Системата актуализира принадлежността на бележката и я премества в избраната папка.

- **Изтриване на бележка**

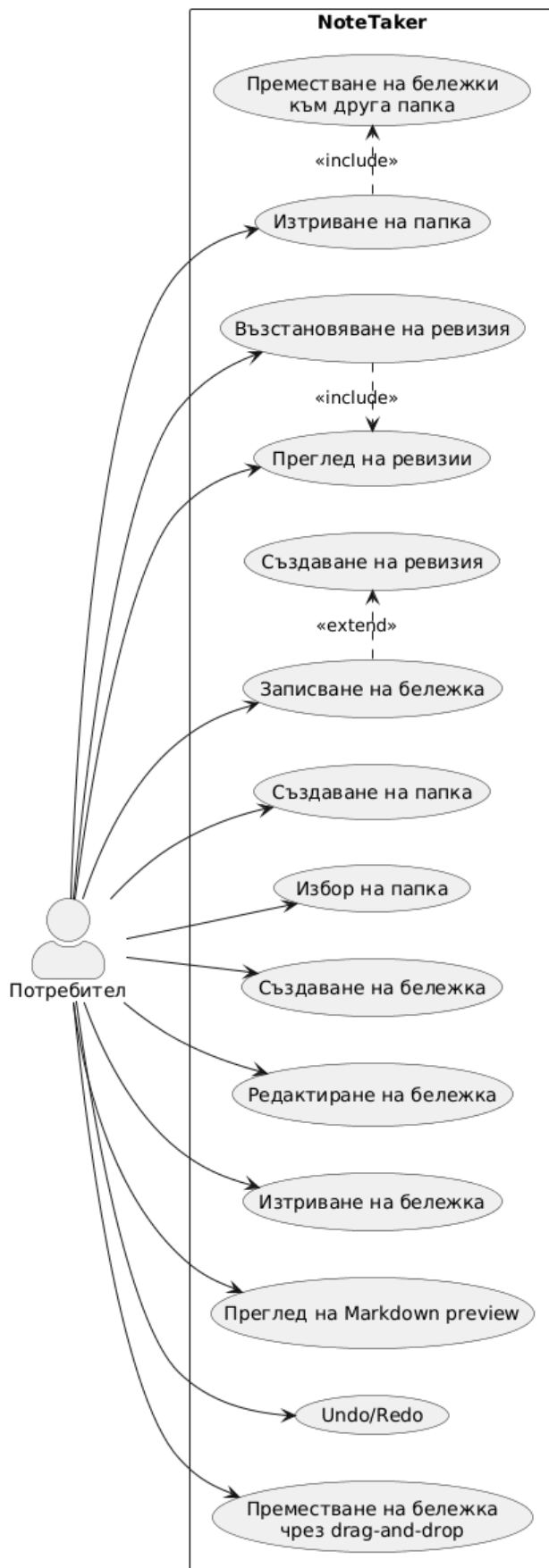
Потребителят избира бележка и стартира операция по изтриване.



След потвърждение системата изтрива бележката и обновява списъка.

2.3.3. Обобщение на use case модела

Use case моделът показва, че приложението е ориентирано към един основен актьор - потребителя, който извършва всички основни операции по създаване, редактиране, организиране и възстановяване на съдържание. Основните случаи на употреба са свързани с управление на папки и бележки, работа с Markdown, съхранение на история на промените и използване на drag-and-drop механизъм за по-лесна организация на данните.



фиг. 1 Use case диаграма



3. Проектиране на системата

Тук ще разгледам общата архитектура на приложението (**MVVM**), както и основните класове и връзките между тях. Целта на този етап от разработката е да се осигури ясна и подредена структура на софтуера, която да отговаря на поставените функционални изисквания, включително работа с база данни, Markdown, ревизии, undo/redo и drag-and-drop.

3.1. Архитектура на системата

Приложението е реализирано като десктоп приложение, разработена с **.NET MAUI**. Архитектурата е изградена така, че да разделя визуалния интерфейс, бизнес логиката, моделите на данните и достъпа до базата данни. Това разделение позволява по-добра организация на кода и по-удобно бъдещо разширяване на функционалността.

Основните архитектурни слоеве в системата са:

- **View слой** - съдържа визуалните елементи на приложението и отговаря за взаимодействието с потребителя
- **ViewModel слой** - съдържа логиката за управление на състоянието, командите и обработката на потребителските действия
- **Model слой** - съдържа класовете, описващи данните, с които работи системата
- **Service слой** - отговаря за връзката с базата данни и изпълнява операциите по запис, четене, редакция и изтриване на информация

В конкретната реализация архитектурата е организирана около един основен екран, който обединява функционалностите за работа с папки, бележки, Markdown и ревизиите. Потребителят взаимодейства с графичния интерфейс, който чрез data binding комуникира с ViewModel слоя. ViewModel от своя страна използва service класа за достъп до SQLite базата данни.

Тази архитектура осигурява следните предимства:

- ясно разделение на отговорностите
- намаляване на зависимостите между графичния интерфейс и бизнес логиката



- по-лесно разширяване на приложението
- по-добра поддръжка и четимост на кода
- възможност за по-структурирана реализация на изискванията

3.2. Организация на проекта

Проектът е организиран в няколко основни папки и файлове, които съответстват на отделните логически части на системата.

3.2.1. Models

В папката **Models** се намират класовете, които описват данните в системата:

- **Folder** - описва папки
- **Note** - описва бележка
- **NoteRevision** - описва ревизия на бележка

Тези класове се използват както за работа в паметта, така и за създаване и поддръжка на таблиците в SQLite базата данни.

3.2.2 ViewModels

В директорията **ViewModels** се намира **MainViewModel**, който е основният клас за управление на логиката на приложението. Той поддържа списъците с папки, бележки и ревизии, текущо избраните обекти, както и командите за действията на потребителя.

3.2.3 Services

В директорията **Services** се намира класът **NoteDatabase**, който реализира достъпа до SQLite базата данни. В него са дефинирани методи за:

- създаване на таблици
- зареждане на папки
- записване и изтриване на папки
- зареждане, записване и изтриване на бележки
- записване и зареждане на ревизии
- преместване на бележки между папки чрез промяна на **FolderId**



3.2.4 View

Визуалната част на системата е реализирана основно чрез:

- `MainPage.xaml` - описва потребителския интерфейс
- `MainPage.xaml.cs` - съдържа логиката за drag-and-drop и прозореца за потвърждение при изтриване

Интерфейсът е структуриран в няколко визуални панела:

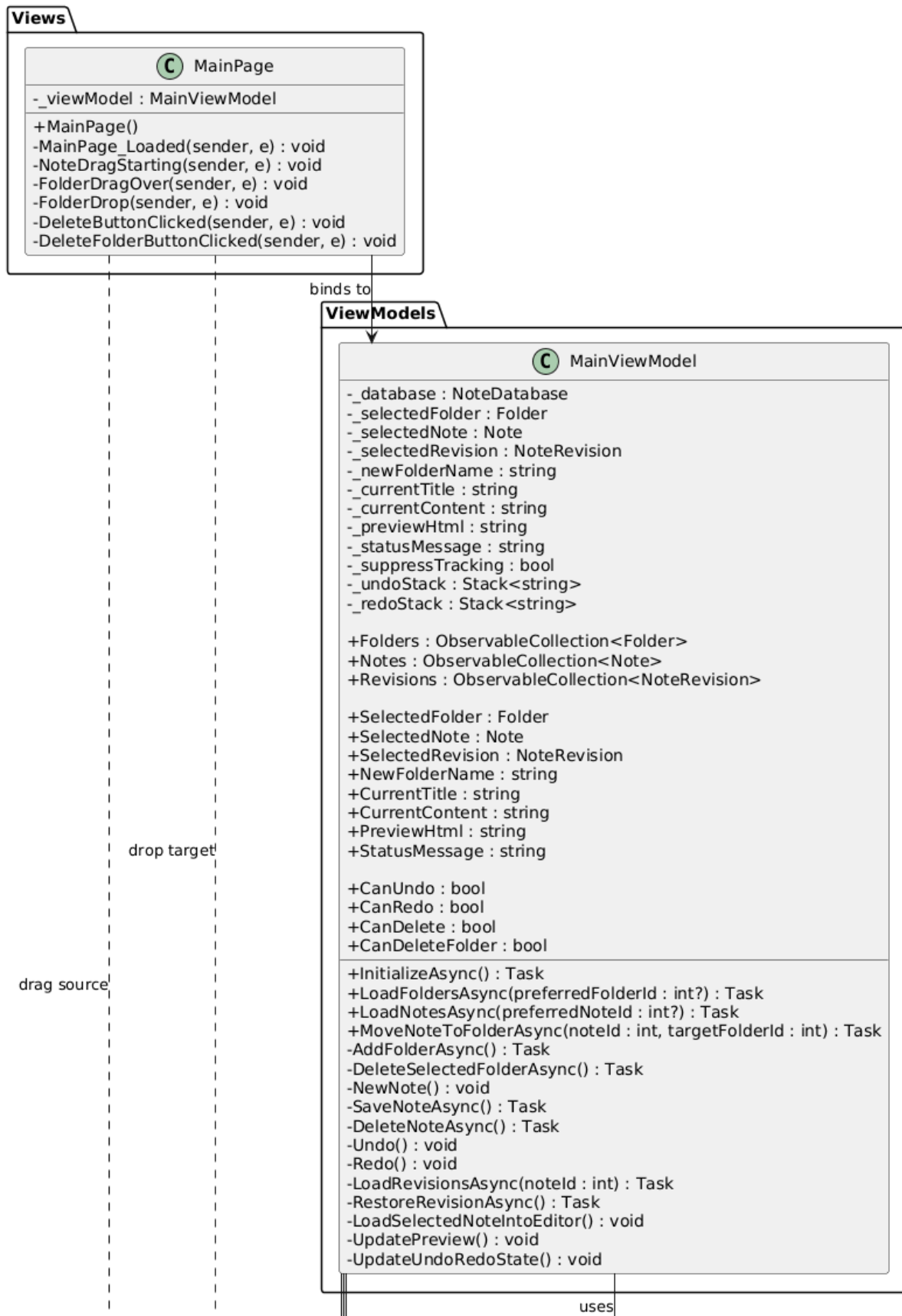
- панел с папки
- панел със списък от бележки
- панел за редакция
- панел за preview
- панел с ревизии

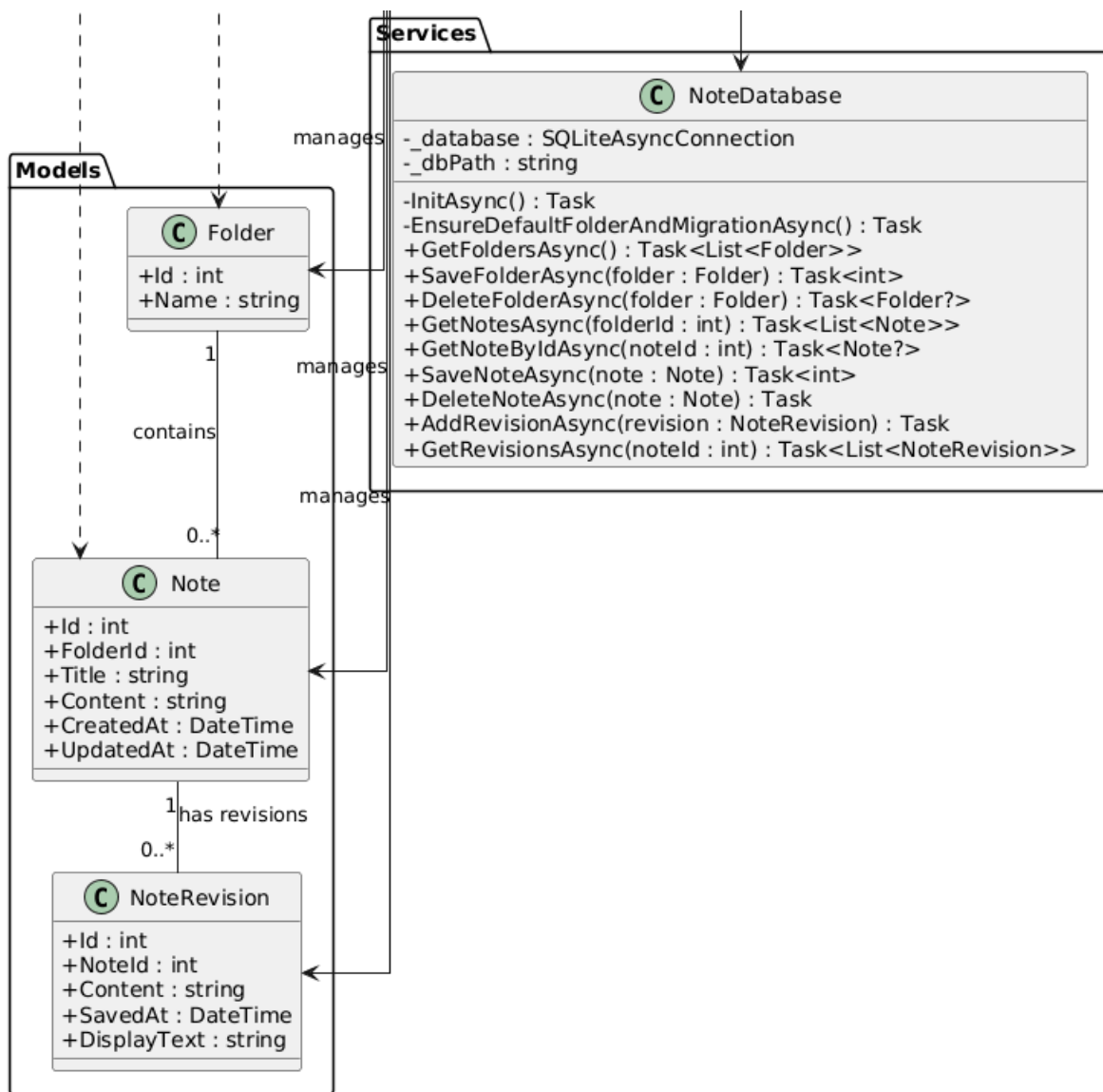
3.2.5 Връзка между View и ViewModel

Връзката между `MainPage` и `MainViewModel` се реализира чрез **BindingContext**. Потребителският интерфейс използва data binding към свойствата на `ViewModel`. Например:

- списъкът с папки е свързан с колекцията `Folders`
- списъкът с бележки е свързан с `Notes`
- текстовите полета са свързани с `CurrentTitle` и `CurrentContent`
- `WebView preview` е свързан с `PreviewHtml`
- бутоните са свързани с команди като `SaveNoteCommand`, `UndoCommand`, `RedoCommand` и други

По този начин всяка промяна в данните автоматично се отразява във визуалния интерфейс





фиг. 2 Class диаграма

3.4. Обяснение на класовата диаграма

В класовата диаграма на системата представя основните класове в приложението и връзките между тях. Тя показва както моделите на данните, така и класовете, отговорни за логиката и визуализацията.

3.4.1 Клас Folder

Класът **Folder** представя папка, в която се групират бележките. Той съдържа следните основни атрибути:



- **Id** - уникален идентификатор
- **Name** - име на папката

Връзката между **Folder** и **Note** е **едно към много**, тъй като една папка може да съдържа множество бележки.

3.4.2 Клас **Note**

Класът **Note** представя отделна бележка. Той съдържа:

- **Id** - уникален идентификатор
- **FolderId** - указва към коя папка принадлежи бележката
- **Title** - заглавие
- **Content** - съдържание
- **CreatedAt** - дата на създаване
- **UpdatedAt** - дата на последна промяна

Всяка бележка принадлежи към точно една папка, но една папка може да съдържа много бележки.

3.4.3 Клас **NoteRevision**

Класът **NoteRevision** представя ревизия на дадена бележка. Той съдържа:

- **Id** - уникален идентификатор
- **NoteId** - идентификатор на бележката, към която принадлежи ревизията
- **Content** - старото съдържание
- **SavedAt** - момент на запис на ревизията.

Връзката между **Note** и **NoteRevision** също е **едно към много**, тъй като една бележка може да има множество предишни версии.

3.4.4 Клас **NoteDatabase**

Класът **NoteDatabase** играе ролята на service слой за достъп до данните. Той поддържа връзката със SQLite базата и предоставя методи за работа с таблиците и обектите в системата. Този клас се използва от



`MainViewModel`, което означава, че между тях съществува зависимост тип **uses**.

Основните отговорности на `NoteDatabase` са:

- инициализация на базата данни
- създаване на таблици
- управление на папки
- управление на бележки
- управление на ревизии
- подпомагане на логиката по преместване и изтриване

3.4.5 Клас `MainViewModel`

`MainViewModel` е основният клас, който управлява логиката на приложението. Той използва `NoteDatabase` и работи с моделите `Folder`, `Note` и `NoteRevision`.

Неговите отговорности включват:

- зареждане на данни
- управление на избора на текуща папка и текуща бележка
- обработка на потребителските команди
- управление на undo/redo стековете
- генериране на Markdown preview
- зареждане и възстановяване на ревизии
- преместване на бележки между папки

3.4.6 Клас `MainPage`

`MainPage` представлява основната визуална страница на приложението. Тя е свързана към `MainViewModel` чрез binding и използва неговите свойства и команди за визуализиране и управление на данните.

`MainPage` съдържа:

- XAML описание на визуалната структура
- code-behind логика за събития, свързани с drag-and-drop
- диалози за потвърждение при изтриване



3.4.7 Основни връзки в класовата диаграма

Класовата диаграма може да бъде обобщена чрез следните връзки:

- Folder 1 → много Note
- Note 1 → много NoteRevision
- MainViewModel използва NoteDatabase
- MainViewModel работи с Folder, Note и NoteRevision
- MainPage е свързана с MainViewModel

Тези връзки показват логическата организация на системата и начина, по който отделните компоненти взаимодействат помежду си.

4. Проектиране на базата данни

Базата данни осигурява постоянно съхранение на папките, бележките и историята на техните промени. Това съответства на поставените изисквания за връзка с БД, редактор със запис в базата и съхранение на ревизии. Използвана е **SQLite** база данни, тъй като работи без необходимост от външен сървър.

4.1. Накратко

Базата данни на приложението съдържа три основни таблици:

- Folder - съхранява информация за папките
- Note - съхранява информация за бележките
- NoteRevision - съхранява историята на промените по бележките

Тази структура е достатъчна, за да реализира логиката на системата по ясен начин. Всеки запис в базата представлява обект - папка, бележка или ревизия на бележка.

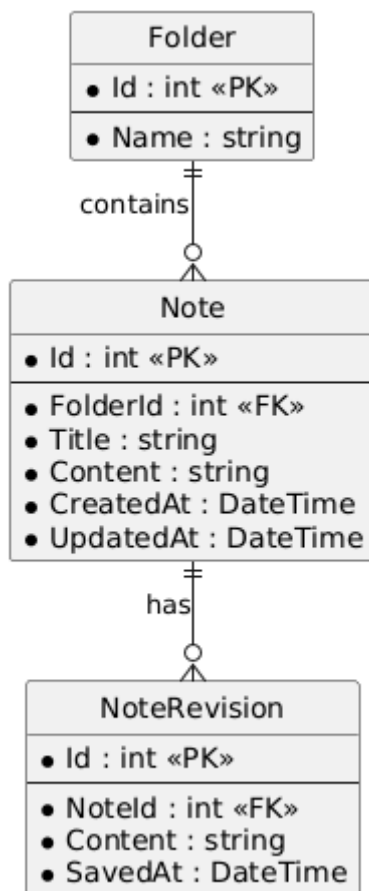
4.2 ER диаграма

ER диаграмата описва основните обекти в системата и връзките между тях. Съществуват две основни зависимости между таблиците:

- една папка може да съдържа много бележки



- една бележка може да има много ревизии



фиг. 3 ER диаграма

Това означава, че всяка бележка принадлежи на точно една папка, а всяка ревизия принадлежи на точно една бележка.

4.2.1 Логика на връзките

Връзката между Folder и Note е реализирана чрез полето FolderId в таблицата Note. Това поле указва към коя папка принадлежи съответната бележка.

Връзката между Note и NoteRevision е реализирана чрез полето NoteId в таблицата NoteRevision. По този начин всяка ревизия е свързана с конкретна бележка и може да бъде заредена при нужда.



4.3 Описание на таблиците

4.3.1 Таблица **Folder**

Таблицата **Folder** съхранява информация за папките. Тя служи за организиране на бележките в отделни категории.

Поleta на таблицата

Поле	Тип	Описание
Id	int	Идентификатор на папката
Name	string	Име на папката

Таблицата позволява създаването и управлението на групи от бележки. Всеки запис е една папка, която може да съдържа множество бележки. При първоначално стартиране на приложението може автоматично да бъде създадена папка по подразбиране, например **General**.

4.3.2 Таблица **Note**

Таблицата **Note** съхранява основната информация за бележките

Поleta

Поле	Тип	Описание
Id	int	Идентификатор на бележката
FolderId	int	Идентификатор на папката, към която принадлежи бележката



Title	string	Заглавие на бележката
Content	string	Съдържание на бележката
CreatedAt	DateTime	Дата и час на създаване
UpdatedAt	DateTime	Дата и час на последна промяна

Тази таблица съдържа текущото състояние на всяка бележка. Така съхраняваме съдържание, заглавието и връзката към папката, в която бележката се намира.

4.3.3 Таблица **NoteRevision**

Таблицата **NoteRevision** съхранява предишните версии на бележките.

Полега

Поле	Тип	Описание
Id	int	Идентификатор на ревизията
NoteId	int	Идентификатор на бележката, към която принадлежи ревизията
Content	string	Старо съдържание на бележката
SavedAt	DateTime	Дата и час на създаване на ревизията



Това позволява съхраняване на история на промените по бележките. При записване на промяна, предишната версия на съдържанието може да бъде запазена като отделна ревизия.

4.4 Цялост на данните

Базата данни трябва да гарантира логическа цялост на информацията.

4.4.1 Цялост при работа с папки

При изтриване на папка нейните бележки не трябва да бъдат изгубени. Вместо това те се преместват в друга съществуваща папка. Така се избягва загуба на данни и се запазва последователността на записите в таблицата **Note**.

4.4.2 Цялост при работа с бележки

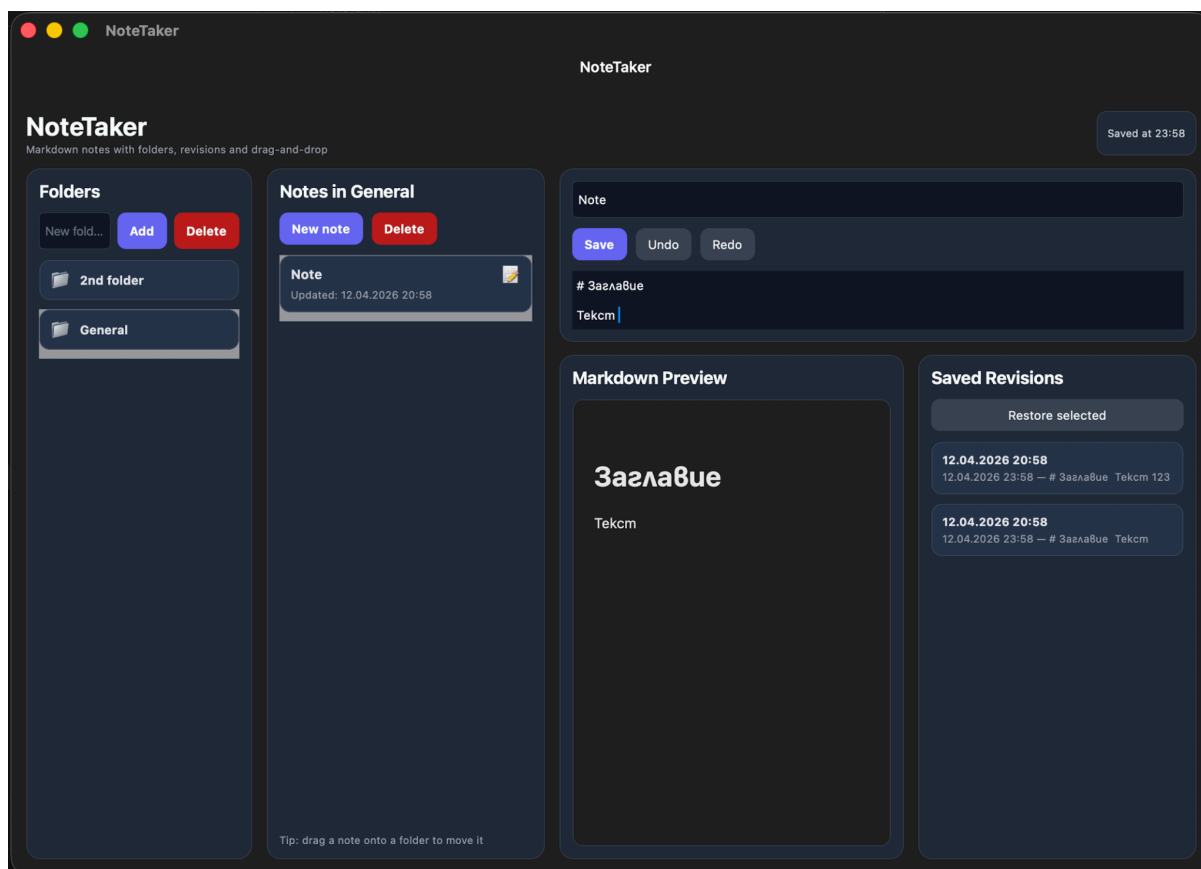
Всяка бележка трябва винаги да бъде свързана с валидна папка. Затова при създаване на бележка се задава текущо избраната папка, а при миграция на стари данни бележките без папка се преместват в папка по подразбиране.

4.4.3 Цялост при работа с ревизии

Всяка ревизия трябва да бъде свързана с реално съществуваща бележка. При изтриване на бележка е необходимо свързаните с нея ревизии също да бъдат премахнати, за да не останат невалидни записи.



5. Имплементация



фиг. 4 Интерфейс

5.1 Управлението на папки

Функционалността за работа с папки е реализирана с цел потребителят да може да организира бележките си в групи. Папките се визуализират в отделен панел в лявата част на интерфейса, което позволява бърз достъп до тях и лесно превключване между различни групи от бележки.

5.1.1 Създаване на папки

Потребителят може да създаде нова папка чрез въвеждане на име и натискане на бутон за добавяне. При изпълнение на тази операция системата проверява дали въведеното име не е празно и дали вече не съществува папка със същото име. Ако данните са валидни, новата папка се записва в базата данни и списъкът с папки се обновява.



5.1.2 Избор на папка

При избор на папка от списъка, системата автоматично зарежда бележките, които принадлежат към нея. Това се реализира чрез ViewModel слоя и презареждане на списъка с бележки според нейния идентификатор.

5.1.3 Изтриване на папки

Реализирана е и функционалност за изтриване на папки. За да се избегне загуба на информация, при изтриване на папка нейните бележки не се премахват, а автоматично се преместват в друга съществуваща папка. Освен това системата не позволява изтриване на последната останала папка. По този начин се гарантира целостта на данните и се предотвратява оставането на бележки без валидна принадлежност.

5.2 Работа с бележки

5.2.1 Създаване на бележка

Потребителят може да създаде нова бележка чрез бутон за създаване. При тази операция интерфейсът подготвя празен редактор и задава начални стойности за заглавие и съдържание. След записване новата бележка се съхранява в базата данни и се свързва с текущо избраната папка.

5.2.2 Редактиране на бележка

При избор на бележка от списъка нейните данни се зареждат в редактора. Потребителят може да променя заглавието и съдържанието, като направените редакции се отразяват в ViewModel слоя чрез data binding.

5.2.3 Записване на бележка

Записването на бележка се осъществява чрез отделна команда. Ако бележката вече съществува, се актуализират нейното заглавие, съдържание и дата на последна промяна. Ако бележката е нова, се създава нов запис в таблицата Note. След успешно записване списъкът с бележки се обновява, а статусното съобщение информира потребителя за извършената операция.



5.2.4 Изтриване на бележка

При изтриване записът се премахва от базата данни, а ако към него има свързани ревизии, те също се изтриват. След това списъкът с бележки се презарежда.

5.3 Реализация на Markdown редактора

За редактиране на съдържанието на бележките е използван Markdown подход. Вместо rich text редактор, системата работи с обикновен текстов редактор, в който потребителят въвежда Markdown синтаксис.

Този подход е избран, защото е лесен за съхранение в база данни и позволява ясно разделение между текстовото съдържание и неговата визуализация.

Чрез Markdown потребителят може да въвежда:

- заглавия
- списъци
- удебелен и наклонен текст
- цитати
- кодови блокове
- хоризонтални линии
- линкове

Редакторът е реализиран чрез стандартен компонент за текстово въвеждане, свързан със свойство във ViewModel слой. По този начин всяка промяна в текста се отразява веднага върху текущото състояние на бележката.

5.4 Реализация на preview функционалността

За да може потребителят да вижда как ще изглежда форматираното съдържание, е реализиран отделен панел за preview.

5.4.1 Преобразуване на Markdown към HTML

Markdown текстът, въведен в редактора, се преобразува в HTML чрез библиотеката **Markdig**. Това преобразуване се извършва във ViewModel слой, където текущото съдържание се обработва и се генерира HTML код.



5.4.2 Визуализация на preview

Полученият HTML се подава към **WebView**, който визуализира крайния резултат. Така потребителят може едновременно:

- да редактира текста в Markdown формат
- да вижда неговия форматиран вид

Тази функционалност значително подобрява удобството при работа и прави приложението по-близко до реални съвременни note-taking системи.

5.5 Реализация на ревизиите

5.5.1 Създаване на ревизия

При запис на съществуваща бележка системата проверява дали съдържанието е променено. Ако има разлика между старото и новото съдържание, предишната версия се записва като нов обект от тип **NoteRevision** в базата данни.

По този начин всяка значима промяна може да бъде проследена и възстановена по-късно.

5.5.2 Зареждане на ревизии

При избор на бележка се зареждат всички ревизии, свързани с нея. Те се визуализират в отделен панел, което позволява на потребителя да разглежда историята на съдържанието.

5.5.3 Възстановяване на ревизия

При избор на конкретна ревизия и изпълнение на команда за възстановяване, съдържанието на избраната ревизия се зарежда обратно в редактора. След това потребителят може да го запише като нова текуща версия на бележката.

5.6 Реализация на undo/redo механизма

Undo/redo механизмът е реализиран във ViewModel слоя чрез два стека:

- **undo стек** - съхранява предишните състояния на текста



- **redo стек** - съхранява отменени състояния, които могат да бъдат възстановени

5.6.1 Undo

При промяна в съдържанието предишното състояние се записва в undo стека. Когато потребителят избере команда Undo, последното състояние се извлича от undo стека, текущото се премества в redo стека и съдържанието се възстановява към предишната стойност.

5.6.2 Redo

При команда Redo системата извлича състояние от redo стека и го възстановява като текущо съдържание, като при това предходното текущо състояние се връща обратно в undo стека.

5.7 Реализация на drag-and-drop

Drag-and-drop функционалността е реализирана с цел по-интуитивно организиране на бележките между папките.

5.7.1 Източник на drag операцията

Бележките в списъка могат да бъдат влачени от потребителя. При започване на drag операцията се предава идентификаторът на съответната бележка.

5.7.2 Drop target

Папките в списъка функционират като drop target. При пускане на бележка върху дадена папка, системата получава идентификатора на бележката и папката, върху която е извършено действието.

5.7.3 Преместване на бележката

След успешен drop системата актуализира полето `FolderId` на бележката и я записва отново в базата данни. След това списъкът с бележки се обновява, за да се отрази преместването.



5.7.4 Значение на функционалността

Drag-and-drop подобрява потребителското изживяване, тъй като позволява бързо и естествено организиране на съдържанието без необходимост от допълнителни бутони или диалогови прозорци.